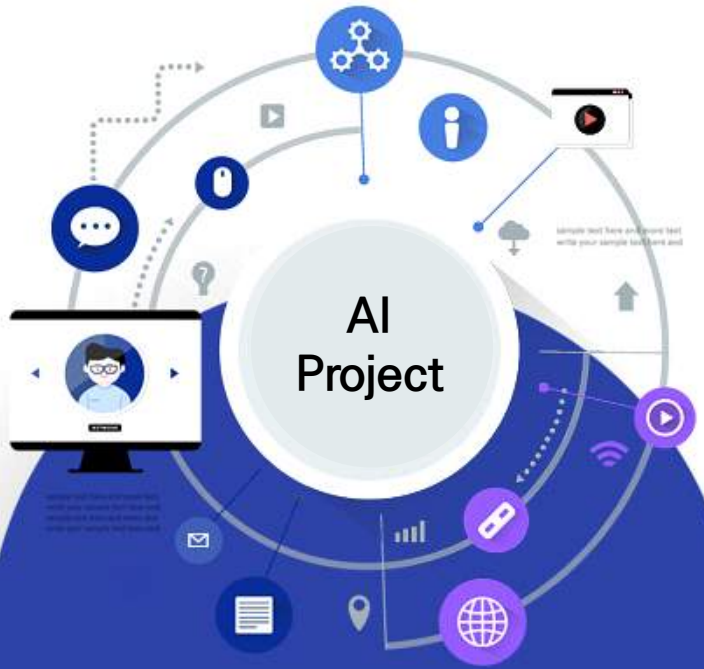
# 심장마비 가능성 예측 모델 개발

작성자 : 최효빈



# 1. 개요 및 현황

| 사업과제 | AI기반 심장마비 가능성 예측모델 개발 및 시각화 |
| --- | --- |
| 추진배경 | · 📌 프로젝트 배경<br>1. 심장질환은 전 세계 주요 사망 원인으로, 조기 예측을 통한 예방이 필수적임.<br>2. 의료 데이터와 AI 기술이 발전하면서, 딥러닝 기반 건강 예측 모델의 필요성이 증가.<br>3. 기존의 심장 질환 예측 연구는 많지만, 해석 가능성과 실시간 활용성에서 한계가 있음. |
| 현재 상황 | · 📌 현재<br>1. 대규모 심장질환 데이터셋(예: UCI Heart Disease Dataset)을 활용한 연구가 활발함.<br>2. SHAP 등 설명 가능한 AI 기법이 의료 분야에서 점점 중요해지고 있음.<br>3. 심장 질환 예측 모델이 있지만, 의료진과 환자가 쉽게 이해할 수 있는 시각화는 부족함. |

# 1. 개요 및 현황

| 사업과제 | AI기반 심장마비 가능성 예측모델 개발 및 시각화 |
| --- | --- |
| 기대효과 | . 📌 기대 효과<br>1. 딥러닝을 활용해 심장마비 위험도를 높은 정확도로 예측 가능.<br>2. SHAP 기반의 해석 가능한 AI를 적용하여 의료진이 예측 결과를 신뢰할 수 있음.<br>3. 직관적인 시각화 도구를 통해 환자도 자신의 건강 상태를 쉽게 파악 가능. |

# 2. 과제 범위

| 과제구분 | | 내용 |
|---|---|---|
| AI | AI기반 심장마비 가능성 예측모델 개발 | 원시 데이터 수집 및 데이터셋 구축 |
| | | 데이터 결측치, 기초 통계량 확인, 표준화, 상관관계 분석 (EDA도구 활용) |
| | | LSTM모델 구축 및 형태 변환, 학습 |
| | | binary_crossentropy, accuracy 평가지표를 활용한 모델 성능 평가 |
| | | 예측 및 평가,ROC Curve 및 AUC 계산 |
| | | 웹 API 및 프로토타입 구축 |
| | | 예측모델 웹기반 시스템 구축 |
| | | 테스트 |
| 시각화 | 심장마비 예측 및 시각화 | 웹 프로토타입 (streamlit) 구축 |
| | | 실제값과 예측값 시각화, 혼동행렬 및 주요 지표 대시보드 |
| | | 모델 성능 비교 |
| | | 통합테스트 및 시운전 |

# 3. 과제 추진 방법 – 구축 대상 선정 기준

**데이터 접근성 및 활용성**

- 데이터 수집 및 관리의 용이성
- Kaggle에서 이미 구축된 데이터베이스 활용 여부
- 종속변수에 영향을 미치는 다양한 독립변수에 대한 정보 포함여부를 통한 모델학습의 유용성

**예측모델 개발 효율성**

- 모델 학습 및 평가 과정 간소화를 위한 다른 건강데이터에 비해 변수가 상대적으로 단순한 구조 여부
- 개발된 모델을 통해 다른 건강 기초 데이터에 적용가능 여부

**문제 해결 기여도 및 경제성**

- 예측모델을 통해 건강관리에 상대적 기여도가 높은 지 여부(ex.  예측 및 대응, 생존확률 증가 등)
- 운영 효율성을 높여 비용절감 효과 여부
- 심장마비라는 건강문제 해결을 통한 사회적 비용감소 효과 여부

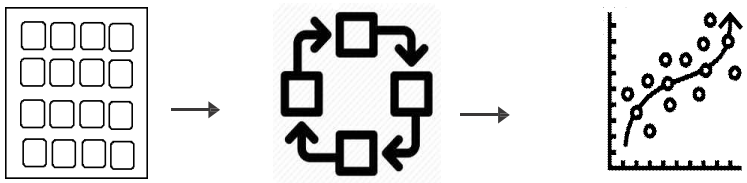
# 3. 과제 추진 방법 – AI 예측 분석모델 적용 대상

| 분류 | 수집데이터 | 예측모델인자(독립변수) | AI예측 분석 대상 |
|---|---|---|---|
| 심장마비 가능성 | • 건강 데이터(외부, 내부)<br>• 성별, 흉통수, 공복 혈당 외에 심장마비와 관련된 핵심변수가 포함된 데이터셋<br>• 1025명의 동일한 항목의 건강 데이터로 테스트 | • 건강변수 (age 외 12개)<br>• age(나이), cp(흉통유형),thalach(휴식 혈압) ,slope(운동 강도) 등 | • 심장마비 가능 여부 (target)<br>• 0: 가능성 낮음<br>• 1: 가능성 높음 |

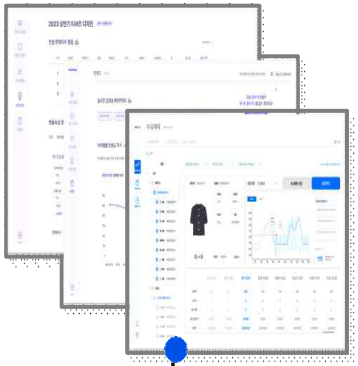
# 3. 과제 추진 방법 – AI 예측 분석모델 구축 프로세스

DATA IMPORTING

AUTO PREDICTION




USER INSIGHT



- 건강 데이터 수집 및 임포팅
- 성별, 흉통유형, 공복혈당 외 심장마비와 관련된 핵심변수 포함

- 변수별 상관관계 분석
- 데이터 표준화 및 전처리(이상치, 결측치 처리)
- 시계열 데이터를 고려하여 학습, 테스트 데이터 분리

- 모델 비교 및 최적 모델 도출(ex. LSTM)
- 시간적 추이/변화에 따른 추론이 가능한 시계열 모델 활용
- 모델 정의 및 학습

- 예측값 계산
- 심장마비 가능성 예측

- 13개 항목의 건강데이터를 통한 심장마비 가능성 분석
- 자신의 가능성을 알고 미리 약을 처방받거나 예방하는 등 의료기술의 향상

# 4. 주요 결과물 – 데이터 수집

## 데이터 수집

- train_data: 303명의 건강 데이터(엑셀)
- test_data: 1025명의 건강 데이터(엑셀)

| age | sex | cp | trestbps | chol | fbs | restecg | thalach | exang | oldpeak | slope | ca | thal | target |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 63 | 1 | 3 | 145 | 233 | 1 | 0 | 150 | 0 | 2.3 | 0 | 0 | 1 | 1 |
| 37 | 1 | 2 | 130 | 250 | 0 | 1 | 187 | 0 | 3.5 | 0 | 0 | 2 | 1 |
| 41 | 0 | 1 | 130 | 204 | 0 | 0 | 172 | 0 | 1.4 | 2 | 0 | 2 | 1 |
| 56 | 1 | 1 | 120 | 236 | 0 | 1 | 178 | 0 | 0.8 | 2 | 0 | 2 | 1 |
| 57 | 0 | 0 | 120 | 354 | 0 | 1 | 163 | 1 | 0.6 | 2 | 0 | 2 | 1 |
| 57 | 1 | 0 | 140 | 192 | 0 | 1 | 148 | 0 | 0.4 | 1 | 0 | 1 | 1 |
| 56 | 0 | 1 | 140 | 294 | 0 | 0 | 153 | 0 | 1.3 | 1 | 0 | 2 | 1 |
| 44 | 1 | 1 | 120 | 263 | 0 | 1 | 173 | 0 | 0 | 2 | 0 | 3 | 1 |
| 52 | 1 | 2 | 172 | 199 | 1 | 1 | 162 | 0 | 0.5 | 2 | 0 | 3 | 1 |
| 57 | 1 | 2 | 150 | 168 | 0 | 1 | 174 | 0 | 1.6 | 2 | 0 | 2 | 1 |
| 54 | 1 | 0 | 140 | 239 | 0 | 1 | 160 | 0 | 1.2 | 2 | 0 | 2 | 1 |
| 48 | 0 | 2 | 130 | 275 | 0 | 1 | 139 | 0 | 0.2 | 2 | 0 | 2 | 1 |
| 49 | 1 | 1 | 130 | 266 | 0 | 1 | 171 | 0 | 0.6 | 2 | 0 | 2 | 1 |
| 64 | 1 | 3 | 110 | 211 | 0 | 0 | 144 | 1 | 1.8 | 1 | 0 | 2 | 1 |
| 58 | 0 | 3 | 150 | 283 | 1 | 0 | 162 | 0 | 1 | 2 | 0 | 2 | 1 |
| 50 | 0 | 2 | 120 | 219 | 0 | 1 | 158 | 0 | 1.6 | 1 | 0 | 2 | 1 |
| 58 | 0 | 2 | 120 | 340 | 0 | 1 | 172 | 0 | 0 | 2 | 0 | 2 | 1 |
| 66 | 0 | 3 | 150 | 226 | 0 | 1 | 114 | 0 | 2.6 | 0 | 0 | 2 | 1 |
| 43 | 1 | 0 | 150 | 247 | 0 | 1 | 171 | 0 | 1.5 | 2 | 0 | 2 | 1 |
| 69 | 0 | 3 | 140 | 239 | 0 | 1 | 151 | 0 | 1.8 | 2 | 2 | 2 | 1 |
| 59 | 1 | 0 | 135 | 234 | 0 | 1 | 161 | 0 | 0.5 | 1 | 0 | 3 | 1 |
| 44 | 1 | 2 | 130 | 233 | 0 | 1 | 179 | 1 | 0.4 | 2 | 0 | 2 | 1 |
| 42 | 1 | 0 | 140 | 226 | 0 | 1 | 178 | 0 | 0 | 2 | 0 | 2 | 1 |

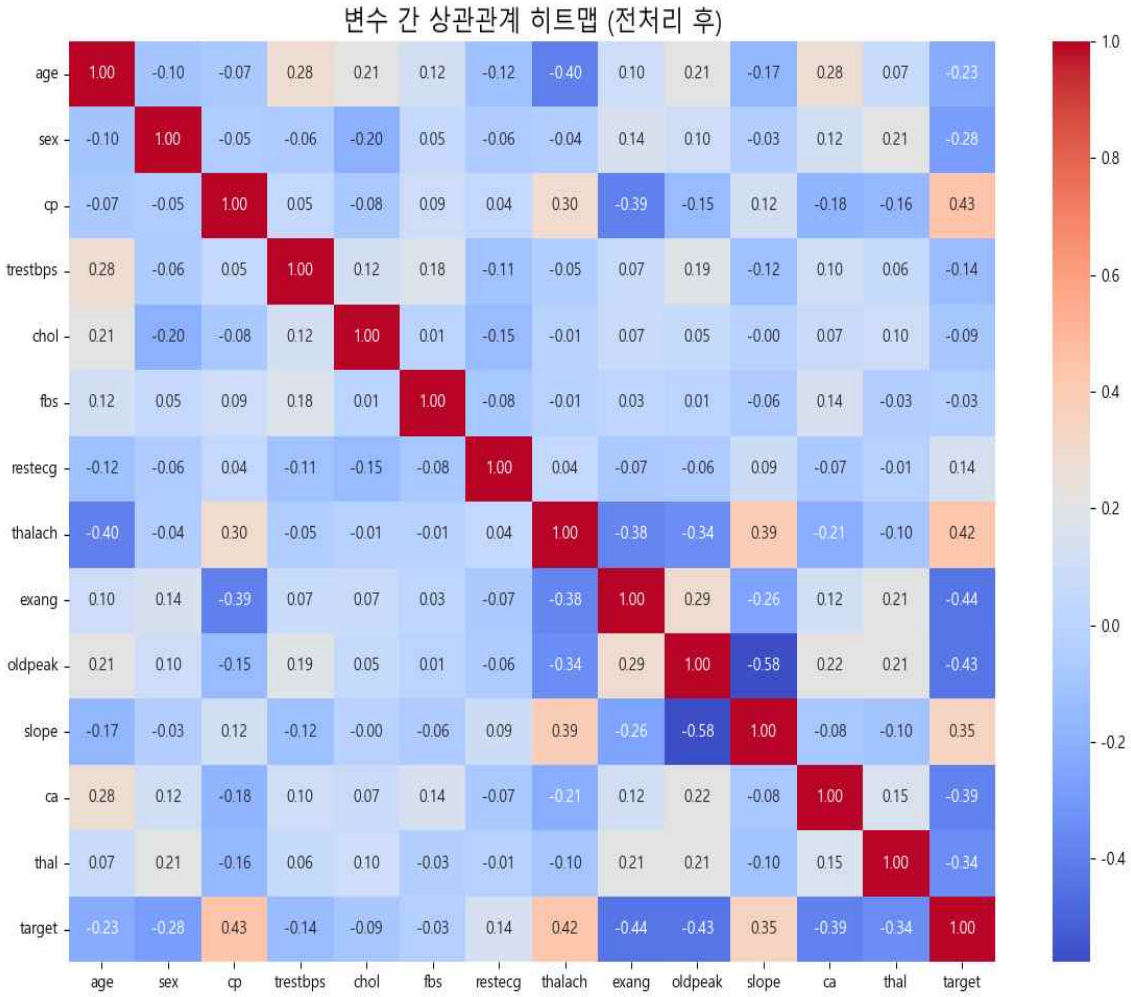
# 4. 주요 결과물 – 데이터 수집

## 데이터 수집

1) age: 나이
2) sex: 성별
3) cp: chest pain type (4 values) 흉통유형 4개의 값
4) trestbps: resting blood pressure 휴식 혈압
5) chol: serum cholestoral in mg/dl 혈청 콜레스테롤(mg/dl)
6) fbs: fasting blood sugar 〉120 mg/dl 공복혈당 //120 이상이면 당뇨병 1, 아니면 0
7) restecg: resting electrocardiographic results (values 0,1,2) 휴식 심전도 결과값
8) thalach: maximum heart rate achieved 달성된 최대 심박수
9) exang : exercise induced angina 운동 유발 협심증
10) oldpeak : ST depression induced by exercise relative to rest 휴식에 비해 운동으로 유발된 ST 저하
11) slope: the slope of the peak exercise ST segment 피크 운동 ST 세그먼트의 기울기
12) ca: number of major vessels (0–3) colored by flourosopy 형광 염색으로 채색된 주요 혈관 수(0–3)
13) thal: 0 = normal; 1 = fixed defect; 2 = reversable defect thal: 0 = 정상; 1 = 고정 결함; 2 = 가역적 결함
14) target: 0 = 심장마비 가능성 낮음 1 = 심장마비 가능성 높음

# 4. 주요 결과물 – 데이터 분석

## 건강데이터 상관관계(Heatmap)




```python
#히트맵
plt.figure(figsize=(12, 8))
sns.heatmap(train_df_viz.corr(), annot=True,
            cmap='coolwarm', fmt='.2f')
plt.title('변수 간 상관관계 히트맵 (전처리 후)', fontsize=16)
plt.tight_layout()
plt.show()
```

- EDA 히스토그램, 히트맵 변수별 분포를 통해 정규분포 여부, 데이터 변환(ex. 로그변환) 필요성 및 변수 간 관계를 유추
- 모델링을 위한 대상 : 심장마비 여부 (target)
- 심장마비에 높은 영향을 미치는 요인 분석
  - 건강변수 :age, cp,thalach,slope 외 10종
- Heatmap 결과로 높은 영향성
  - cp(흉통유형): 0.43
  - thalach(최대 심박수): 0.42

# 4. 주요 결과물 – 탐색적 데이터 분석(EDA)

## 건강데이터의 결측값 및 중복값 통계

```python
import os
import pandas as pd
from ydata_profiling import ProfileReport

#데이터 불러오기
file_path = 'dataset/train_heart.csv'
df = pd.read_csv(file_path)

#YData Profiling으로 데이터 프로파일링 보고서 생성
profile = ProfileReport(df,title="heartattack Profiling Report", explorative=True)

#보고서를 html파일로 저장
output_dir = 'report'
output_file = os.path.join(output_dir, 'heart_report.html')
profile.to_file(output_file)
print(f"프로파일링 보고서가 생성되었습니다.: {output_file}")
```

✓ 7.2s

```
100%|██████| 14/14 [00:00<00:00, 499.99it/s]00:00, 69.58it/s, Describe variable: target]
Summarize dataset: 100%|██████| 48/48 [00:02<00:00, 17.07it/s, Completed]
Generate report structure: 100%|██████| 1/1 [00:03<00:00,  3.35s/it]
Render HTML: 100%|██████| 1/1 [00:00<00:00,  1.29it/s]
Export report to file: 100%|██████| 1/1 [00:00<00:00, 124.91it/s]
프로파일링 보고서가 생성되었습니다.: report\heart_report.html
```

Overview | Alerts 5 | Reproduction

**Dataset statistics**

| | |
|---|---|
| Number of variables | 14 |
| Number of observations | 303 |
| Missing cells | 0 |
| Missing cells (%) | 0.0% |
| Duplicate rows | 1 |
| Duplicate rows (%) | 0.3% |
| Total size in memory | 33.3 KiB |
| Average record size in memory | 112.4 B |

**Variable types**

| | |
|---|---|
| Numeric | 5 |
| Categorical | 9 |

- 독립변수 : 13개(Numeric)
- 종속변수: 1개(int)
- 결측값 : 0개
- 중복값: 1개(0.3%)

```python
#결측치 확인
print("Train 데이터 결측치 확인")
print(train_df.isnull().sum())
print("\nTest 데이터 결측치 확인")
print(test_df.isnull().sum())
```

```
Train 데이터 결측치 확인
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
thalach     0
exang       0
oldpeak     0
slope       0
ca          0
thal        0
target      0
dtype: int64
```

```
Test 데이터 결측치 확인
age         0
sex         0
cp          0
trestbps    0
chol        0
fbs         0
restecg     0
...
slope       0
ca          0
thal        0
dtype: int64
```

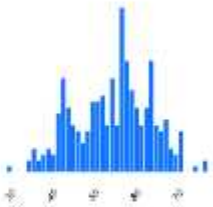
# 4. 주요 결과물 – 탐색적 데이터 분석(EDA)

## 주요 변수별 데이터 분포(Histogram)

### age

Real number (ℝ)

| | | | |
|---|---|---|---|
| Distinct | 41 | Minimum | 29 |
| Distinct (%) | 13.5% | Maximum | 77 |
| Missing | 0 | Zeros | 0 |
| Missing (%) | 0.0% | Zeros (%) | 0.0% |
| Infinite | 0 | Negative | 0 |
| Infinite (%) | 0.0% | Negative (%) | 0.0% |
| Mean | 54.366337 | Memory size | 2.5 KiB |



More details

### cp

Categorical

High correlation

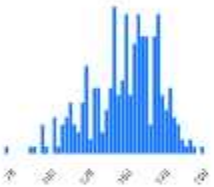
| | |
|---|---|
| Distinct | 4 |
| Distinct (%) | 1.3% |
| Missing | 0 |
| Missing (%) | 0.0% |
| Memory size | 17.3 KiB |

More details

### thalach

Real number (ℝ)

| | | | |
|---|---|---|---|
| Distinct | 91 | Minimum | 71 |
| Distinct (%) | 30.0% | Maximum | 202 |
| Missing | 0 | Zeros | 0 |
| Missing (%) | 0.0% | Zeros (%) | 0.0% |
| Infinite | 0 | Negative | 0 |
| Infinite (%) | 0.0% | Negative (%) | 0.0% |
| Mean | 149.646865 | Memory size | 2.5 KiB |

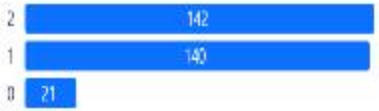


More details

### slope

Categorical

| | |
|---|---|
| Distinct | 3 |
| Distinct (%) | 1.0% |
| Missing | 0 |
| Missing (%) | 0.0% |
| Memory size | 17.3 KiB |



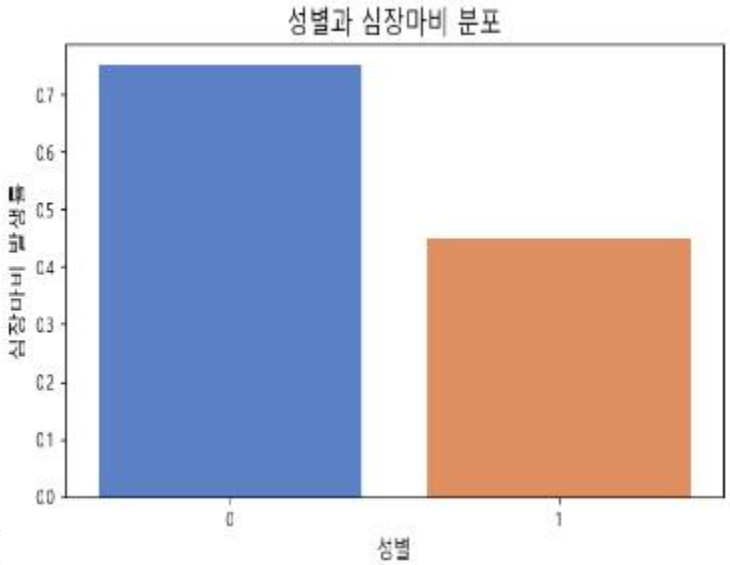

More details

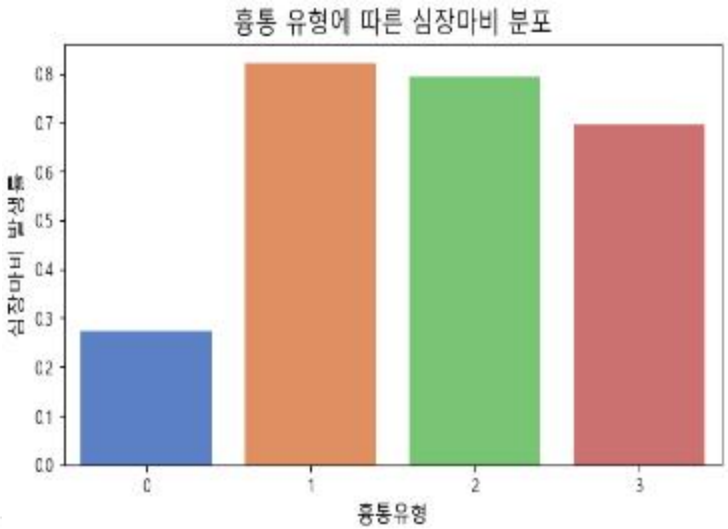
# 4. 주요 결과물 – 기초 통계량

## 성별과 심장마비 관련 통계량 확인

```
#데이터 기초 통계량 확인
plt.figure(figsize=(8, 4))
sns.barplot(x='sex',y='target',data=train_df, palette='muted',errorbar=None)
plt.title('성별과 심장마비 분포', fontsize=15)
plt.xlabel('성별', fontsize=12,)
plt.ylabel('심장마비 발생률', fontsize=12)
plt.show()
plt.figure(figsize=(8, 4))
sns.barplot(x='cp',y='target',data=train_df, palette='muted',errorbar=None)
plt.title('흉통 유형에 따른 심장마비 분포', fontsize=15)
plt.xlabel('흉통유형', fontsize=12)
plt.ylabel('심장마비 발생률', fontsize=12)
plt.show()
```

- Heatmap에서 높은 영향성이 있는 피쳐들을 barplot을 사용하여 확인
- 성별 0: 여자 , 1: 남자
  =〉 여자의 확률이 0.7 이상으로 높은 연관성
- 흉통은 1번 유형이 0.8로 제일 높은 연관성



# 4. 주요 결과물 – 데이터 전처리

## 데이터 스케일링 및 분리

```
# 스케일링 (MinMaxScaler)
scaler = MinMaxScaler()
X = train_df.drop('target', axis=1)
y = train_df['target']
X = scaler.fit_transform(X)
test_data = scaler.transform(test_df)
```

- MinMaxScaling을 통해 데이터를 0~1범위로 정규화, 모델 학습에 적합한 형태로 변환
- X = target을 제외한 모든 독립변수
- y = target 의 종속변수
- fit_transform 을 통해 훈련 데이터에 대해 스케일링 규칙을 학습

```
# train, validation set 분리
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

- 학습 데이터 80%, 테스트 데이터 20%로 구분

```
# LSTM 입력을 위한 reshape (samples, time steps, features)
X_train = X_train.reshape(X_train.shape[0], 1, n_features)
X_val = X_val.reshape(X_val.shape[0], 1, n_features)
test_data = test_data.reshape(test_data.shape[0], 1, n_features)
```

- LSTM 입력을 위해서 3차원 형태로 변환 (samples, time steps, features)
- 샘플개수, 1, 변수 개수

# 4. 주요 결과물 – LSTM 세팅 및 모델 구축

## 모델 정의 및 컴파일

```python
# 2. LSTM 모델 구축
model = Sequential()
model.add(LSTM(64, input_shape=(1, n_features), return_sequences=True))
model.add(Dropout(0.2))
model.add(LSTM(32, return_sequences=False))
model.add(Dropout(0.2))
model.add(Dense(1, activation='sigmoid'))  # 심장마비 대한 이진 분류(1, 0)이므로 sigmoid 사용

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
# binary_crossentropy : 이진 분류에 사용 (ex. 0, 1 등)
```

```python
#모델 구조 확인
model.summary()
```

✓ 0.0s

Model: "sequential_2"

| Layer (type) | Output Shape | Param # |
|---|---|---|
| lstm_4 (LSTM) | (None, 1, 64) | 19,968 |
| dropout_4 (Dropout) | (None, 1, 64) | 0 |
| lstm_5 (LSTM) | (None, 32) | 12,416 |
| dropout_5 (Dropout) | (None, 32) | 0 |
| dense_2 (Dense) | (None, 1) | 33 |

- layer 순차적으로 쌓는 Sequentail()모델 선언
- <u>첫번째 레이어</u>
  64개 유닛,입력 데이터(time step 1개, feature n개)
- 과적합 방지 드롭아웃 0.2
- <u>두번째 레이어</u>
  32개 유닛
- 과적합 방지 드롭아웃 0.2
- Dense(1): 출력 뉴런 1개 (이진 분류)
- activation='sigmoid': 확률값(0~1)을 반환하도록 설정

- adam : 최적화 알고리즘
  binary_crossentropy: 이진 분류
  accuracy : 정확도

---

- 첫번째 레이어층 = 19,968
- 두번째 레이어층 = 12,416
- Dense층 : 입력뉴런 32 +1

# 4. 주요 결과물 – 데이터 학습

## 모델학습

```python
# 3. 모델 학습
epochs = 100
batch_size = 32

history = model.fit(
    X_train, y_train,
    epochs=epochs,
    batch_size=batch_size,
    validation_data=(X_val, y_val),
    callbacks=[early_stopping]
)
```

✓ 6.7s

학습 멈춤 이후 에폭 수: 34

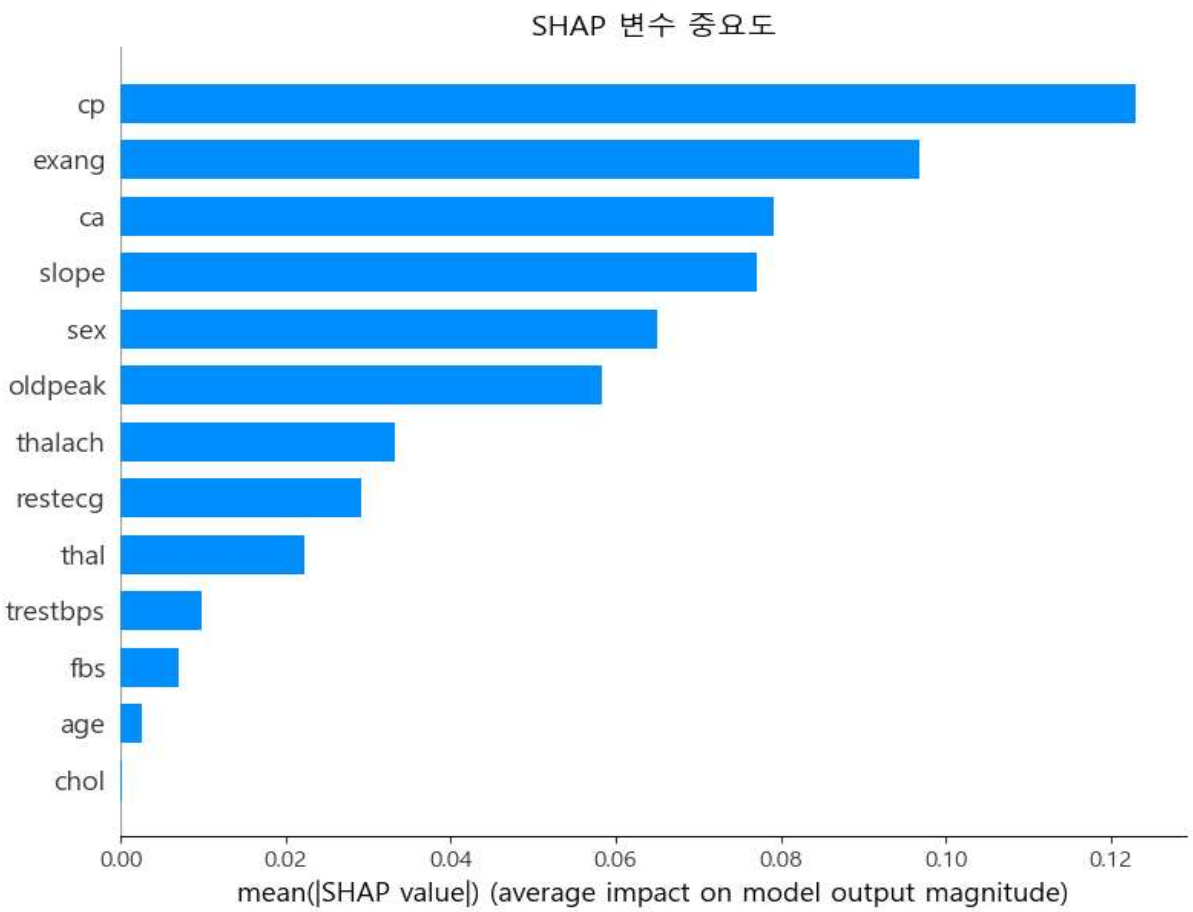
```
Epoch 1/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 3s 59ms/step - accuracy: 0.5718 - loss: 0.6920 - val_accuracy: 0.5902 - val_loss: 0.6887
Epoch 2/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.6358 - loss: 0.6874 - val_accuracy: 0.6066 - val_loss: 0.6845
Epoch 3/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6253 - loss: 0.6823 - val_accuracy: 0.6885 - val_loss: 0.6791
Epoch 4/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6231 - loss: 0.6787 - val_accuracy: 0.6885 - val_loss: 0.6718
Epoch 5/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6883 - loss: 0.6697 - val_accuracy: 0.7049 - val_loss: 0.6613
Epoch 6/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6938 - loss: 0.6595 - val_accuracy: 0.7705 - val_loss: 0.6469
Epoch 7/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6818 - loss: 0.6531 - val_accuracy: 0.8033 - val_loss: 0.6271
Epoch 8/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7883 - loss: 0.6202 - val_accuracy: 0.8197 - val_loss: 0.5997
Epoch 9/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7953 - loss: 0.5909 - val_accuracy: 0.8033 - val_loss: 0.5653
Epoch 10/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.7960 - loss: 0.5625 - val_accuracy: 0.8361 - val_loss: 0.5251
Epoch 11/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7837 - loss: 0.5414 - val_accuracy: 0.8525 - val_loss: 0.4884
Epoch 12/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - accuracy: 0.7872 - loss: 0.5055 - val_accuracy: 0.8525 - val_loss: 0.4511
```

- 모델 학습 횟수 100번
- 데이터 셋 32개로 나누어 학습
- .fit을 통해서 모델 학습
- ‘early_stopping’ 10번으로 설정, 학습 개선이 없으면 중단

- 중단 에폭수 34

# 4. 주요 결과물 – 변수 중요도

## 변수 중요도




- SHAP을 통한 변수 중요도
- 1. cp: 흉통 유형
- 2. exang: 운동 유발 협심증
- 3. ca: 주요 혈관의 수

# 4. 주요 결과물 – 시각화

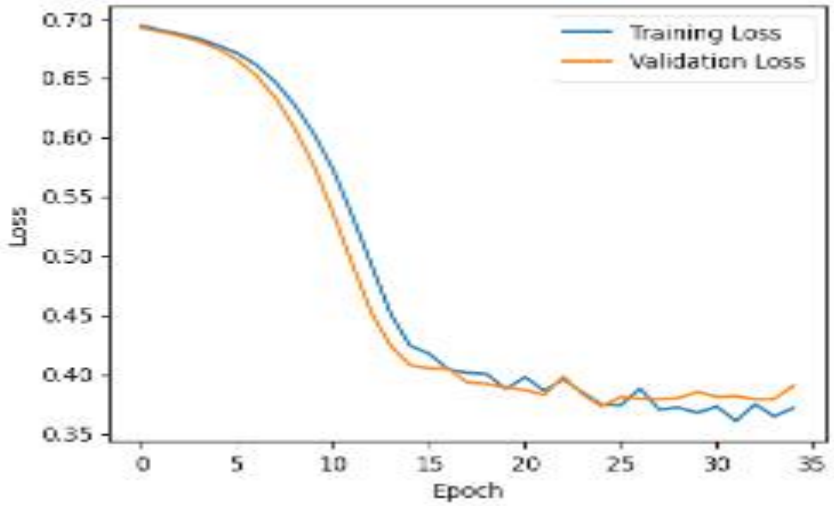
## 모델학습 및 시각화

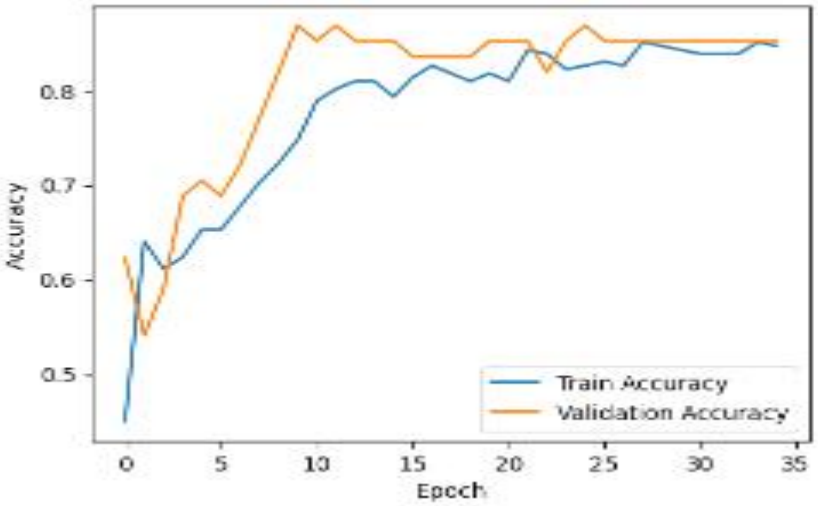
```
#4. 예측 및 평가
y_pred_val_proba = model.predict(X_val, verbose=0)
y_pred_val = (y_pred_val_proba > 0.5).astype(int)
#테스터 데이터 예측
y_pred_proba = model.predict(test_data, verbose=0)
y_pred = (y_pred_proba > 0.5).astype(int)
```

- 이진 분류에 맞춰서 예측 및 평가
- 학습데이터의 예측 확률이 0.5 이상이면 1로 반영

```
# ROC Curve 및 AUC 계산 (Validation set)
fpr, tpr, thresholds = roc_curve(y_val, y_pred_val_proba)
roc_auc = auc(fpr, tpr)
```

- roc_curve 함수를 사용하여 ROC Curve를 계산
- auc 함수를 사용하여 ROC Curve 아래 면적 계산



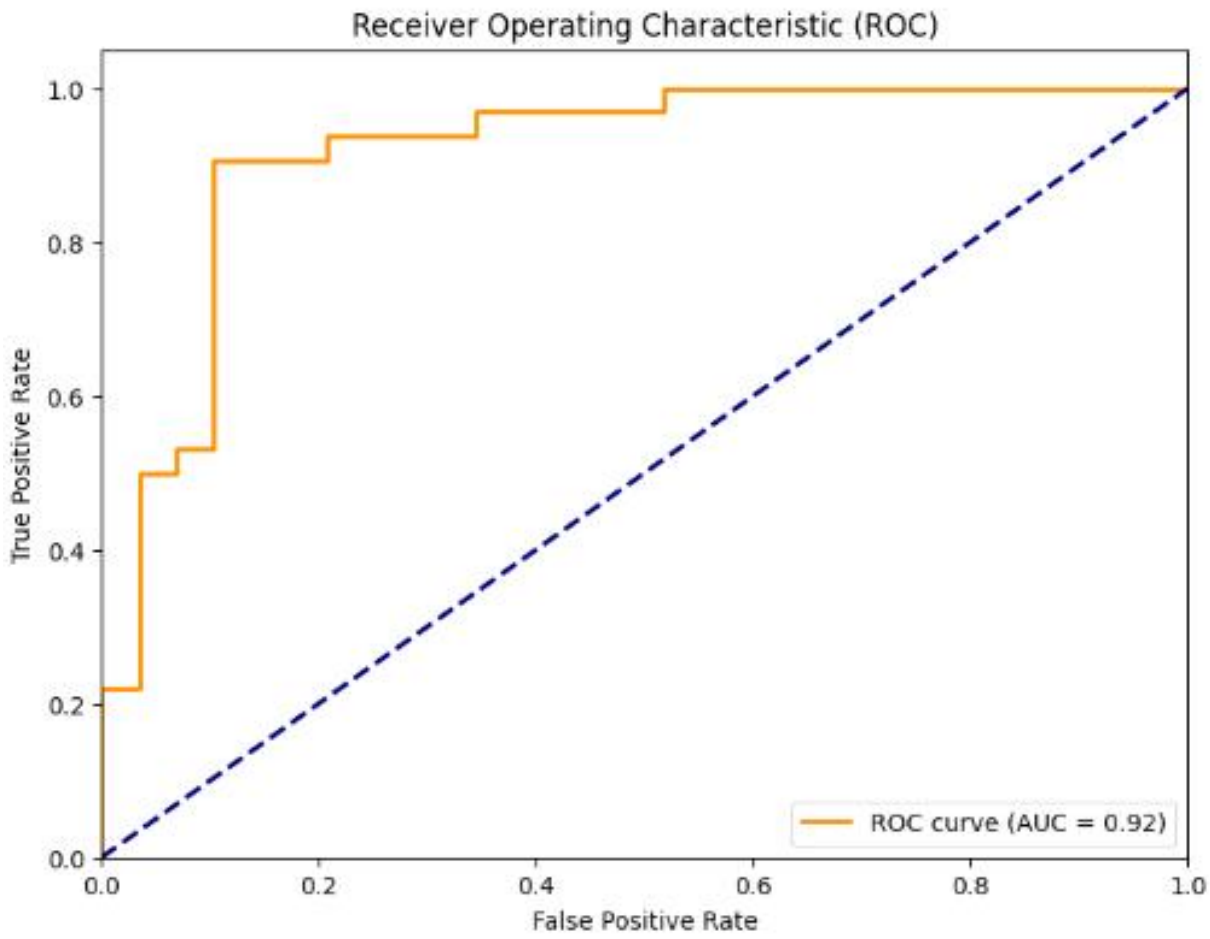




- 초기 학습손실은 좋은 결과
- 에폭이 10~15이후 loss가 줄어들다가 높아지는 경우는 과적합이 될 수도 있음

- 정확도는 초기에 높게 우상향하며 좋은결과
- 에폭 20이후 부터는 정체되는 현상 =〉 과적합 의심

# 4. 주요 결과물 – 시각화

## ROC Curve 시각화

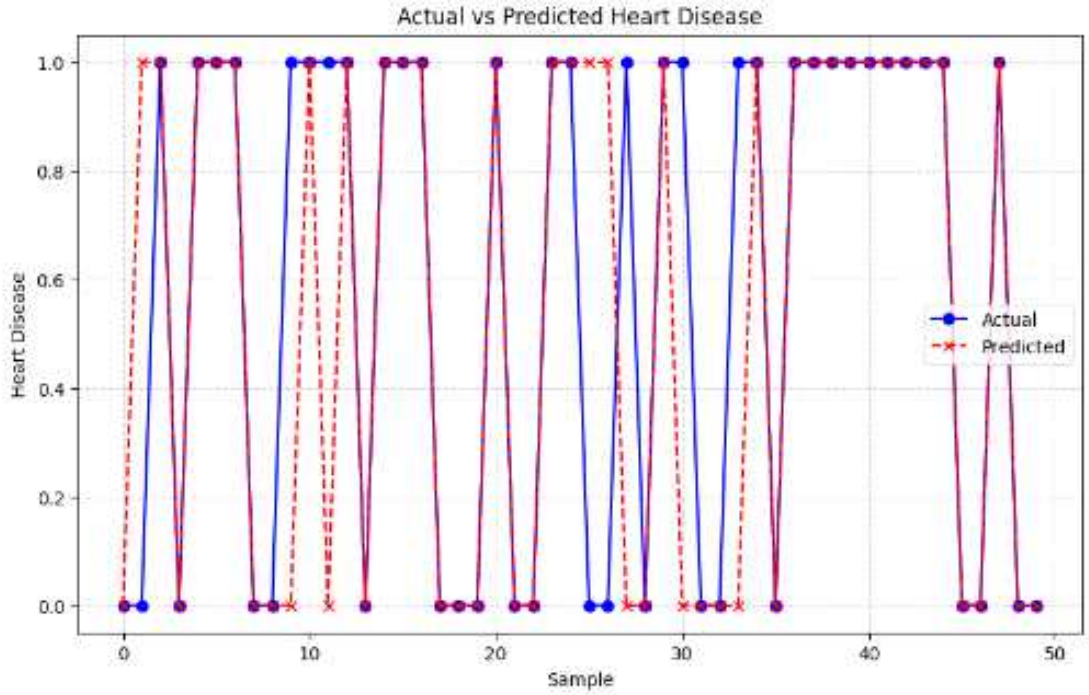



- ROC 곡선은 이진 분류 모델의 성능을 평가
- AUC:ROC곡선 아래의 면적의미
  0.92 =〉 훌륭한 모델 =〉 이진 분류에 적합한 성능
- 곡선이 왼쪽 모서리 위와 가까워 양성클래스와 음성 클래스 예측 비율이 좋음

# 4. 주요 결과물 – 시각화

## 실제값 VS 예측값

```
#실제값 vs 예측값 시각화
plt.figure(figsize=(10, 6))
plt.plot(y_val[:50].values, label='Actual', color='blue', marker='o')
plt.plot(y_pred_val[:50].flatten(), label='Predicted',
         color='red', linestyle='--',marker='x')
plt.xlabel('Sample')
plt.ylabel('Heart Disease')
plt.title('Actual vs Predicted Heart Disease')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```



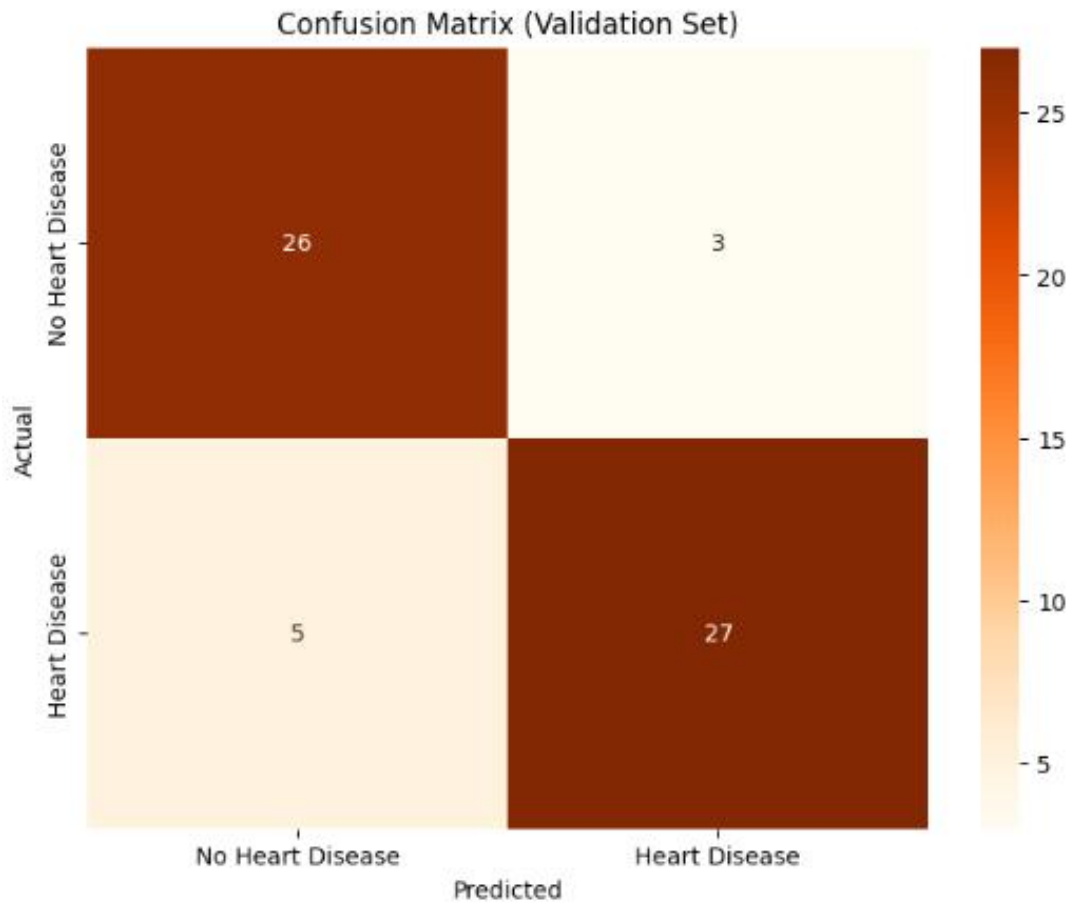

- 가로축 : 0~50까지 샘플
- 세로축 : 심장질환의 유무 (1: 심장마비)
- 파란 실선 : 실제값
- 빨간 점선 : 예측값

---

- 샘플 기준
- 초반 샘플 (0~10)
  예측값이 실제값의 변화를 따라가려함
- 중간 샘플 (10~40)
  예측값과 실제값이 80% 유사
- 후반샘플(40~50)
  예측값과 실제값이 100% 동일

# 4. 주요 결과물 – 시각화

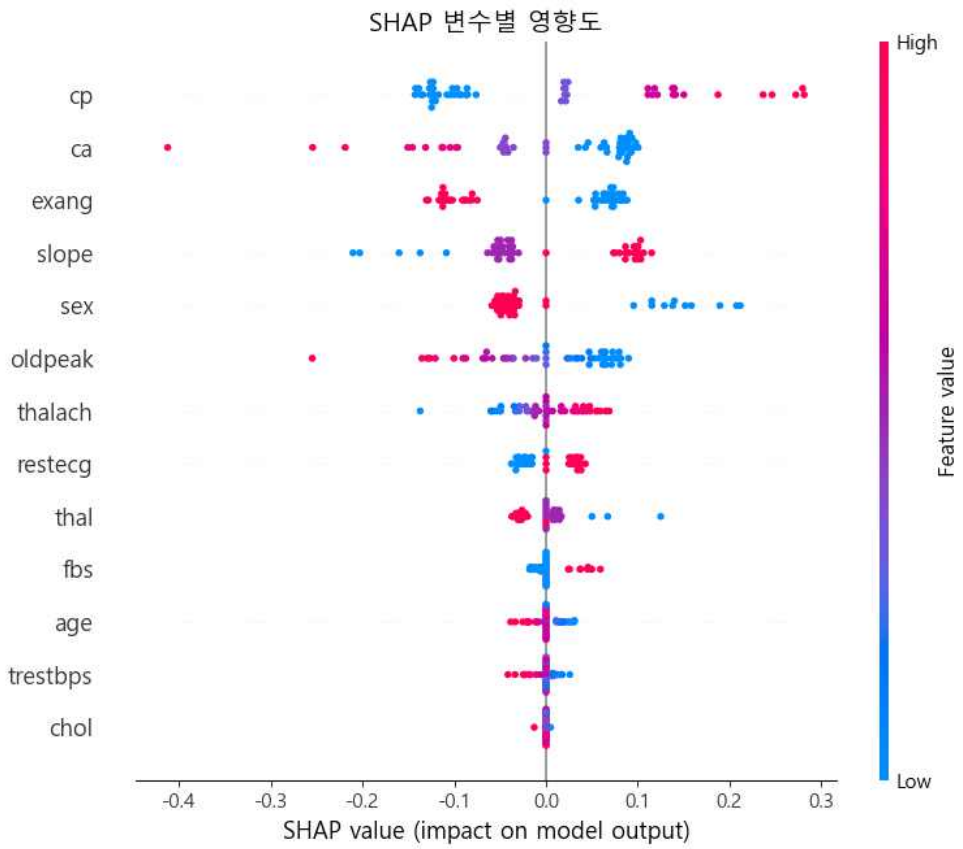
## 혼동 행렬




- 혼동행렬 => 분류 성능 평가
- 심장질환 있음 =27
- 심장질환 없음 =26
- 잘못예측 = 3,5
- 정확도
  Accuracy = (TP + TN) / 총 샘플 수 = (27 + 26) / 61 = 53 / 61 ≈ 0.8689 또는 86.89%
- 정밀도
  Precision = TP / (TP + FP) = 27 / (27 + 3) = 27 / 30 = 0.9 또는 90%

# 4. 주요 결과물 – 시각화

## SHAP 값




- 각 모델이 심장마비에 긍정, 부정적으로 어떤 영향을 미쳤는지 알수 있는 시각화 그래프
- cp: 높은 수치가 양수에 위치 =〉 결과에 영향도 높음
- ca, oldpeak : 높은 수치가 음수에 위치
  =〉 질병이 없다고 예측가능

# 4. 주요 결과물 – 모델 성능 비교

## 모델 성능 비교

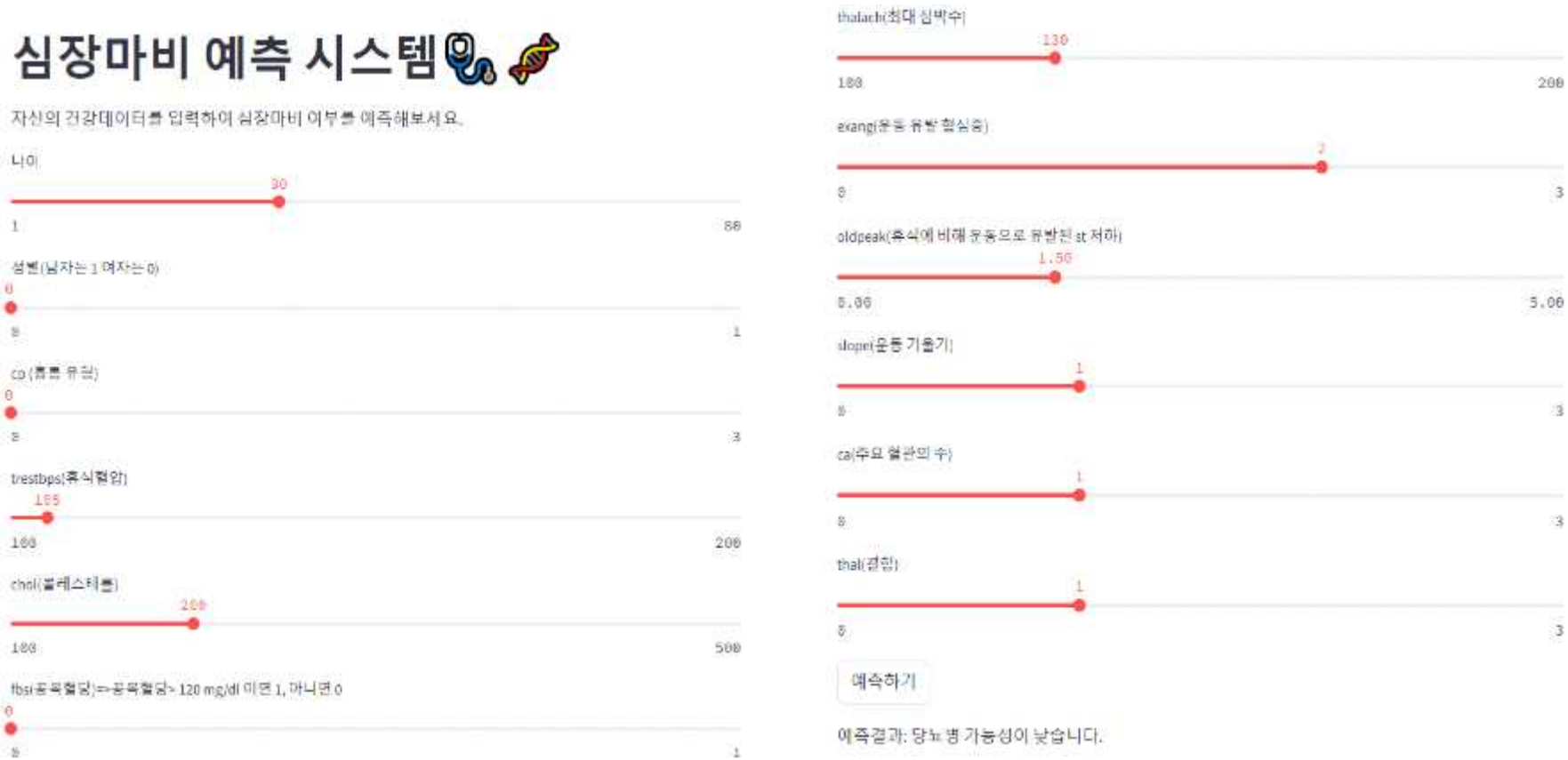
```
# Classification Report (Validation set)
print("\nClassification Report (Validation Set):\n",
      classification_report(y_val, y_pred_val))
```

```
Classification Report (Validation Set):
              precision    recall  f1-score   support

           0       0.81      0.90      0.85        29
           1       0.90      0.81      0.85        32

    accuracy                           0.85        61
   macro avg       0.85      0.85      0.85        61
weighted avg       0.86      0.85      0.85        61
```

- 정밀도, 재현율, 평균, 샘플수
  심장마비 가능성 0 : 정밀도 81%
  심장마비 가능성 1 : 정밀도 90%
- 총 정확도는 85%
- macro avg(거시평균) : 85%
- weighted avg(가중 평균) :85%

# 4. 주요 결과물 – 모델 성능 비교

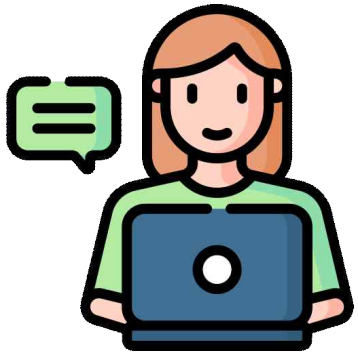
## streamlit



# 5. 마치며

## 소감 및 아쉬운점

1. 지난번 보험료 예측하는 회귀모델에서 분류모델로 해봤는데, train과 test로 나누어서 분류 모델로 데이터 선정은 만족
2. 시간이 있었더라면 학습의 수치를 조절하면서 좀더 오차율을 줄일수 있으면 좋았을텐데 하는 아쉬움이 남음
3. 하지만 다양한 시각화를 통해 분석하고 언제 어떻게 사용해야하는지 알게돼서 보람있음
4. 작은 과제지만, 이런 AI 모델이 실생활에 도움이 돼서 심장질환으로 사망하는 사망률을 줄일 수 있다면 좋겠다고 생각함



# 감사합니다